

# 5054 Homework 4

LAN Tianwei  
tlanaa@connect.ust.hk

November 20, 2025

## Problem 1: Basic Knowledge

### 1. Equivalent representation of maximum margin classifier

Let  $(\beta, \beta_0)$  be any feasible hyperplane. Its geometric margin is

$$\gamma = \frac{1}{\|\beta\|} \min_i y_i(\beta^\top x_i + \beta_0).$$

Define the normalized parameters  $\tilde{\beta} = \beta/\|\beta\|$  and  $\tilde{\beta}_0 = \beta_0/\|\beta\|$ , so that  $\|\tilde{\beta}\| = 1$ . Then

$$\min_i y_i(\tilde{\beta}^\top x_i + \tilde{\beta}_0) = \frac{1}{\|\beta\|} \min_i y_i(\beta^\top x_i + \beta_0) = \gamma.$$

Thus, setting  $M = \gamma$ , the triple  $(\tilde{\beta}, \tilde{\beta}_0, M)$  satisfies  $\|\tilde{\beta}\| = 1$ ,  $y_i(\tilde{\beta}^\top x_i + \tilde{\beta}_0) \geq M$  for all  $i$ , and achieves objective value  $M = \gamma$ .

Conversely, any feasible point  $(\beta, \beta_0, M)$  of the constrained problem satisfies  $\|\beta\| = 1$  and  $y_i(\beta^\top x_i + \beta_0) \geq M$  for all  $i$ , so the geometric margin of  $(\beta, \beta_0)$  is at least  $M$ .

Therefore, maximizing the geometric margin is equivalent to maximizing  $M$  under the given constraints.

### 2. Equivalent representation of SVC

The linear support vector classifier (SVC) is defined by the decision function:

$$f(x) = \beta_0 + \beta^\top x.$$

In the dual formulation of the SVC, we express the weight vector  $\beta$  as a linear combination of the training samples:

$$\beta = \sum_{i=1}^n a_i y_i x_i,$$

where  $a_i \geq 0$  are the Lagrange multipliers associated with the constraints.

Substituting this into the decision function gives:

$$f(x) = \beta_0 + \left( \sum_{i=1}^n a_i y_i x_i \right)^\top x = \beta_0 + \sum_{i=1}^n a_i y_i \langle x_i, x \rangle.$$

By the Karush-Kuhn-Tucker (KKT) conditions, for any non-support vector  $x_i$ , the corresponding multiplier  $a_i = 0$ . Only for support vectors — those lying on or within the margin boundary — do we have  $a_i > 0$ .

Therefore, we can write the classifier as:

$$f(x) = \beta_0 + \sum_{i=1}^n a'_i \langle x_i, x \rangle,$$

where  $a'_i = a_i y_i$ . Since  $y_i \in \{-1, 1\}$ , the nonzero coefficients  $a'_i$  still occur only for support vectors. Moreover, the values of  $a_i$  (and hence  $a'_i$ ) are determined solely by the inner products  $\langle x_j, x_k \rangle$  and the labels  $y_j$  through the dual optimization problem.

Thus, the linear support vector classifier can be represented as stated

### 3. RBF Kernel's impact on SVM's bias and variance

In the Radial Basis Function (RBF) kernel for SVM, defined as

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2),$$

the parameter  $\gamma > 0$  controls the influence radius of each training point.

#### Effect on Variance:

A large  $\gamma$  value makes the kernel function decay rapidly with distance. This results in a decision boundary that is highly flexible and sensitive to local variations in the training data — effectively fitting the training set very closely. Consequently, the model exhibits **high variance**, which may lead to overfitting, especially if the training data contains noise or outliers.

#### Effect on Bias:

Conversely, a small  $\gamma$  value causes the kernel to have a wider influence region. The resulting decision boundary becomes smoother and more global, potentially ignoring fine details in the data. This leads to **high bias**, as the model may underfit the true underlying pattern.

**Trade-off Summary:** Thus,  $\gamma$  acts as a regularization parameter that balances bias and variance:

- Large  $\gamma \Rightarrow$  Low bias, High variance.
- Small  $\gamma \Rightarrow$  High bias, Low variance.

Selecting an appropriate  $\gamma$  via cross-validation is crucial for achieving good generalization performance.

## 4. Preprocessing before PCA

PCA does not natively handle missing data. It requires a complete, numeric matrix as input. Therefore, if the dataset contains missing values (e.g., NaN or NULL), PCA cannot be applied directly. The user must preprocess the data to fill or remove missing entries before running PCA.

Best Practices for Preprocessing Before PCA:

- **Imputation:** Replace missing values with reasonable estimates. Common methods include:
  - Mean/median imputation (simple but may distort variance).
  - K-nearest neighbors (KNN) imputation (preserves local structure).
  - Iterative imputation (e.g., using regression models or MICE).
- **Remove Rows or Columns:** If missingness is severe (e.g., >50% of values missing in a column or row), consider dropping those features or samples.
- **Scale and Center:** PCA is sensitive to the scale of variables. Always standardize features to have zero mean and unit variance (i.e., use Z-score normalization) unless all features are on the same scale.
- **Check for Outliers:** Outliers can heavily influence PCA directions. Consider robust PCA or outlier detection/removal before applying standard PCA.
- **Consider Robust or Probabilistic PCA:** For datasets with many missing values or outliers, alternatives like Probabilistic PCA (PPCA) or Robust PCA (RPCA) may be more appropriate, as they can model uncertainty or decompose data into low-rank and sparse components.

## 5. K-Means++ method

The K-means++ algorithm is an improved initialization method for the standard K-means clustering algorithm. While standard K-means randomly selects  $K$  initial centroids, K-means++ chooses them in a smart, probabilistic way to reduce the risk of poor clustering results.

K-means++ works as the following steps:

1. Choose the first centroid uniformly at random from the data points.
2. For each subsequent centroid, select a point with probability proportional to its squared distance from the nearest already chosen centroid. This encourages centroids to be spread out across the data space.
3. Repeat step 2 until  $K$  centroids are selected.
4. Run the standard K-means algorithm using these initialized centroids.

**Advantages over Standard K-means:**

- **Better Initialization:** By spacing centroids apart initially, K-means++ avoids placing multiple centroids close together or in empty regions, which often happens with random initialization.

- **Faster Convergence:** The algorithm typically converges in fewer iterations because the starting points are more representative of the underlying cluster structure.
- **Improved Final Clustering Quality:** K-means++ significantly reduces the chance of converging to a poor local optimum, leading to lower within-cluster sum of squares (WCSS) on average.
- **Theoretical Guarantee:** K-means++ provides an expected approximation guarantee — it yields a solution that is  $O(\log K)$ -competitive with the optimal clustering, whereas random initialization offers no such guarantee.

In summary, K-means++ is not a different clustering algorithm but a smarter initialization strategy that makes standard K-means more robust, faster, and more likely to find high-quality clusters.

## Problem 2: Classification on 20 Newsgroups Data

### 1. Random Forest (CV Error: 23.64%)

|           | pred_comp | pred_rec | pred_sci | pred_talk |
|-----------|-----------|----------|----------|-----------|
| true_comp | 4349      | 87       | 82       | 87        |
| true_rec  | 218       | 3058     | 88       | 155       |
| true_sci  | 331       | 117      | 2034     | 175       |
| true_talk | 162       | 137      | 110      | 5052      |

| Keyword    | RF Importance |
|------------|---------------|
| windows    | 0.046907      |
| christian  | 0.037066      |
| car        | 0.035491      |
| god        | 0.035284      |
| government | 0.029720      |
| team       | 0.027125      |
| jews       | 0.023942      |
| graphics   | 0.021183      |
| religion   | 0.020365      |
| space      | 0.019041      |

### 2. Boosting Tree (CV Error: 21.95%)

|           | pred_comp | pred_rec | pred_sci | pred_talk |
|-----------|-----------|----------|----------|-----------|
| true_comp | 4230      | 47       | 155      | 173       |
| true_rec  | 269       | 2773     | 151      | 326       |
| true_sci  | 522       | 93       | 1742     | 300       |
| true_talk | 218       | 80       | 218      | 4945      |

| Keyword    | BT Importance |
|------------|---------------|
| windows    | 0.083766      |
| god        | 0.063075      |
| government | 0.052666      |
| christian  | 0.048759      |
| car        | 0.048355      |
| team       | 0.045763      |
| space      | 0.030254      |
| graphics   | 0.029177      |
| jews       | 0.029148      |
| gun        | 0.026980      |

### 3. Model Comparison

Boosting Tree achieves a lower 5-fold CV misclassification error (21.95%) compared to Random Forest (23.64%), indicating superior generalization performance on this task. The confusion matrices reveal that BT makes more errors in classifying `comp` and `rec` posts, likely due to overlapping vocabulary (e.g., "team", "space"). However, it performs better on `talk` posts, which are often more distinct.

Feature importance analysis shows that both models identify similar keywords as most important (e.g., `windows`, `god`, `car`), but BT assigns significantly higher importance scores to these features, reflecting its sequential, additive nature that focuses on correcting previous errors. This suggests BT is better at capturing subtle patterns in the sparse binary text data.

In conclusion, for this multi-class text classification problem, Boosting Tree outperforms Random Forest, primarily because its boosting mechanism is more effective at learning from the structured, albeit sparse, keyword occurrences.

### 4. Linear Discriminant Analysis (CV Error: 24.18%)

|           | pred_comp | pred_rec | pred_sci | pred_talk |
|-----------|-----------|----------|----------|-----------|
| true_comp | 4102      | 45       | 239      | 219       |
| true_rec  | 341       | 2607     | 181      | 390       |
| true_sci  | 620       | 119      | 1530     | 388       |
| true_talk | 268       | 125      | 299      | 4769      |

### 5. Quadratic Discriminant Analysis (CV Error: 31.69%)

|           | pred_comp | pred_rec | pred_sci | pred_talk |
|-----------|-----------|----------|----------|-----------|
| true_comp | 4444      | 101      | 42       | 18        |
| true_rec  | 625       | 2849     | 16       | 29        |
| true_sci  | 1246      | 322      | 911      | 178       |
| true_talk | 1036      | 1203     | 77       | 3145      |

## 6. Compare All Models

Among the four models evaluated on the 20 Newsgroups subset, Boosting Tree achieves the lowest 5-fold cross-validation misclassification error at 21.95%, outperforming Random Forest (23.64%), LDA (24.18%), and QDA (31.69%).

The confusion matrices reveal that Boosting Tree and Random Forest both excel at identifying the `talk` class with high precision, but Boosting Tree shows slightly better balance across all classes. In contrast, LDA suffers from notable confusion between `comp` and `sci` (239 and 620 misclassifications, respectively), likely due to overlapping technical vocabulary. QDA performs poorly overall, especially in the `sci` and `talk` classes, with 1,246 `sci` posts misclassified as `comp` and 1,203 `talk` posts misclassified as `rec`, indicating that its assumption of class-specific covariance structures is ill-suited to this sparse binary text representation.

In summary, tree-based ensemble methods (particularly Boosting Tree) are better suited for this high-dimensional, discrete text classification task than generative linear/quadratic discriminant models.

## Problem 3: Classification on MNIST Data

### 1. Linear SVM

CV Error: 0.89%, Training Time: 0.43s

|        | pred_3 | pred_6 |
|--------|--------|--------|
| true_3 | 1250   | 12     |
| true_6 | 10     | 1190   |

### 2. RBF SVM (CV Error: 0.37%)

CV Error: 0.37%, Training Time: 16.05s

|        | pred_3 | pred_6 |
|--------|--------|--------|
| true_3 | 1256   | 6      |
| true_6 | 3      | 1197   |

### 3. Performance Comparison

The RBF SVM achieves a lower test error (0.37%) than the linear SVM (0.89%), with fewer misclassifications in both classes. This indicates that the non-linear decision boundary better captures the distinction between digits 3 and 6. However, this gain comes at a higher computational cost: training time increases from 0.43 seconds to 16.05 seconds. Thus, the RBF kernel offers improved accuracy at the expense of efficiency.

## 4. Linear SVM on Digits {1, 2, 5, 8} (4-Class)

The best regularization parameter was selected via 5-fold CV, yielding  $C = 1$ . The model was trained in **52.02 seconds** and evaluated on the 4,806-sample test set.

The final test mis-classification error is **5.85%** (accuracy: 94.15%). The confusion matrix shows strong performance across all classes, with the highest recall for digit 1 (98%) and the lowest for digit 8 (90%). The most frequent confusion occurs between digits 5 and 8 (63 instances of digit 8 misclassified as 5), which is consistent with their visual similarity.

Overall, the linear SVM provides an efficient and accurate solution for this 4-class handwritten digit recognition task.

## 5. Best SVM Model on Full MNIST (10-Class)

The best performance was achieved using a linear SVM with  $C = 10$ . The model was selected via 5-fold cross-validation on the 30,000-sample training set and evaluated on the 12,000-sample test set.

The final model attained a test misclassification error of **8.79%** (accuracy: 91.21%) with a training time of **602.36 seconds** ( 10 minutes). The confusion matrix and classification report indicate strong performance across most digits, with the highest accuracy for digits 1 (98% recall) and 6 (95% recall). The most frequent confusions occur between visually similar digits such as 2/8, 3/8, 5/8, and 5/9, which is consistent with known challenges in MNIST classification.

Overall, the linear SVM with appropriate regularization ( $C = 10$ ) provides a robust and efficient solution for this 10-class handwritten digit recognition task.